Design Google Docs

功能性需求（Functional Requirements）

1. 使用者應該能夠建立文件（documents）。
2. 多位使用者應該能夠同時線上編輯同一份文件（collaborative editing）。
3. 使用者的修改應該能夠即時同步給其他正在編輯同一份文件的使用者（real-time updates）。
4. 使用者必須能即時看到其他協作者的游標位置（collaborator cursors / presence indicators）。

非功能性需求（Non-Functional Requirements）

1. 文件必須達到最終一致性（eventual consistency）→ 所有使用者最終看到的文件內容必須一致。
2. 單一文件最多允許 100 位使用者同時編輯。
3. 更新必須具備低延遲（low latency）。
4. 系統必須能擴展至數十億份文件（billions of documents）以及數百萬名同時在線使用者（millions of concurrent users）。

buildmoat.org

API 設計（API Design）

1. 建立文件（Creating docs）

```
POST v1/docs
{
  title
  ...
}
```

```
-> {
  doc_id
}
```

- `title`（文件標題）：文件的顯示名稱。
- `doc_id`（文件 ID）：系統建立並回傳的唯一識別碼，用於後續編輯與協作。

2. 傳送編輯操作（Sending edits）

透過 WebSocket（雙向即時通訊）連線傳送編輯事件：

```
SEND {
  type:"insert",
  ...
}
```

- `type: "insert"`（插入操作）：在指定位置插入內容。

```
SEND {
  type:"updateCursor",
  position: ...
}
```

- `type: "updateCursor"`（更新游標）：回報使用者目前的游標位置，用於顯示協作者游標。

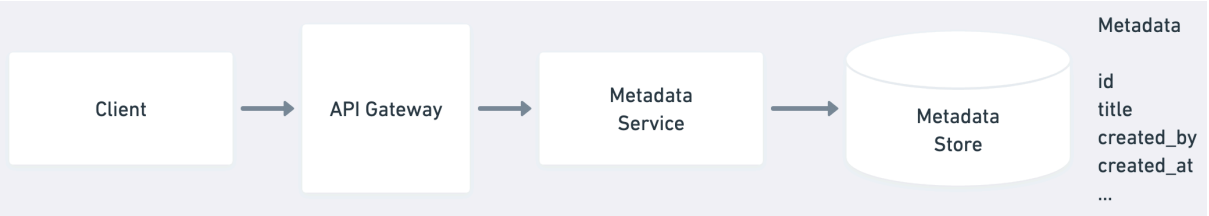
```
SEND {
  type:"delete",
  ...
}
```

- `type: "delete"`（刪除操作）：刪除指定範圍的內容。

High-Level Design

1. 使用者能夠創建文件

buildmoat.org



1. 這是一個簡單的 CRUD service，使用者透過 POST request 建立新文件。
2. Database 的選擇可以很彈性，SQL 或 NoSQL 都能透過 partition 與 replication 來擴展。
    a. 在這裡我們暫時不深入資料庫細節；在 high-level design 中，更重要的是如何支援多位使用者同時進行 concurrent updates。

2. 多個使用者能夠同時線上修改文件

此問題的核心在於：當多位使用者同時間修改同一份線上文件時，如何確保**所有人最後看到的結果一致**，且**每個人的修改都能正確反映**。

Option 1：每次編輯都送出整份快照（Sending snapshots for every edits）

對於每一次編輯，client 都將整份 document snapshot 傳送到 backend。這顯然不可行，原因包括：

1. 對單一文件而言，所有 snapshots 加總的資料量會非常龐大。

    例如：若一份文件有 1000 次編輯、每份 snapshot 為 1KB，僅此一份文件就會產生 1GB 的資料。

2. 衝突解決（conflict resolution）困難：

    若僅以最新 snapshot 作為最終狀態，會遺失其他使用者的修改。

    例如：A 與 B 同時看到一份空白文件並開始輸入，最後只會保留「最後送達的 snapshot」。

Option 2：只送出編輯操作（Sending just the edits）

與其傳送整份 snapshot，我們只傳送編輯操作（edits），例如 `INSERT(3, "H")`、`DELETE(4)`。

這能大幅降低傳輸與儲存成本，但仍有一個關鍵問題：**操作的上下文（context）與順序（order）**。

假設以下情境：

buildmoat.org

```
A 與 B 同時看到 "This is a doc"
A 嘗試在結尾插入 "ument" → INSERT(13, "ument")
B 嘗試在結尾插入 "!"     → INSERT(13, "!")
```

- 若 A 的 edit 先到，結果會是：`"This is a doc!ument"`
- 若 B 的 edit 先到，結果會是：`"This is a document!"`

顯然，結果不一致（雖然第二個才是期望結果）。

在第一種情況下，我們需要將 B 的 edit **轉換（transform）**為 `INSERT(18, "!")`。

這表示 edits 是 **contextual** 的：

- 操作的正確性取決於當下文件狀態
- 操作的順序會影響結果
- 因此需要根據順序對 edits 進行 transformation，以確保最終狀態一致

解決此問題常見的兩種演算法（此處不深入細節，超出 1 小時 SD 面試範圍）：

1. **Operational Transformation（OT，操作轉換）**
   - 需要一個 central server 接收同一份文件的所有 edits
   - Server 先決定唯一順序，再對後到的 edits 進行 transformation 後套用
   - 限制：同一份文件的所有 edits 必須到同一台 server，對併發使用者數有上限
   - Google Docs 採用此方法
2. **Conflict-free Replicated Data Types（CRDTs，無衝突複寫資料型態）**
   - 特殊設計的資料結構，允許多個 replica 在本地獨立修改（即使離線）
   - 只要同步彼此的 updates，即可自動收斂到相同狀態（Strong Eventual Consistency, SEC）
   - 代價：
     - 通常需要 tombstones 或較多 metadata
     - 需等「所有 replica 都看過」後，才能安全進行 GC

經驗法則：

- **強離線、多主、P2P、邊緣節點多、偶爾才連線** → 傾向 **CRDT**
- **有穩定中心服務、需要可控文字意圖與體驗、極高併發編輯** → 多採 **OT（中心化）** 或 OT + CRDT 的 hybrid

buildmoat.org

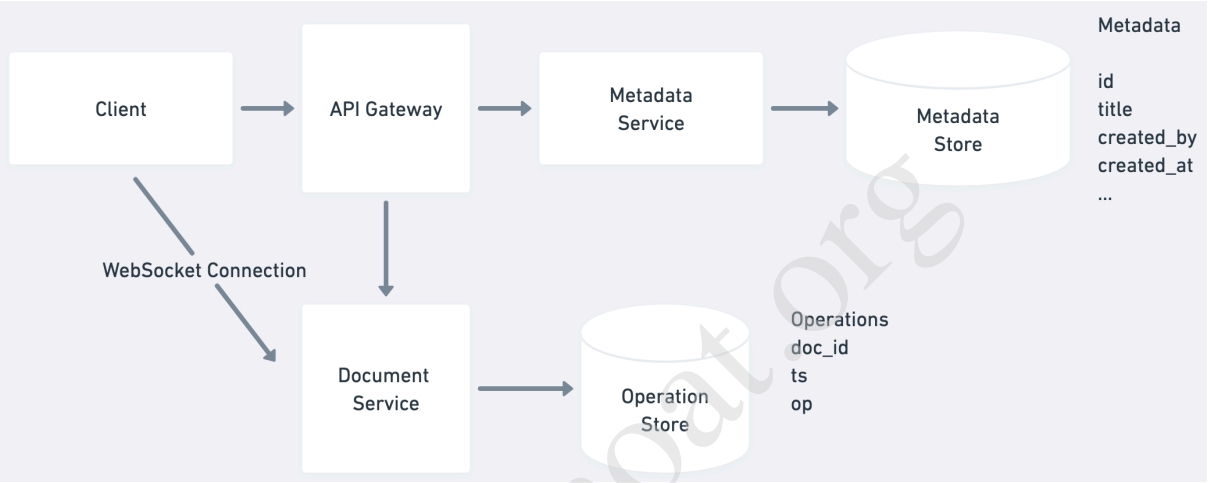
在本設計中，我們選擇 **OT**，因為單一文件的 concurrent users 上限為合理數量（≤100），且 OT 具備以下優勢：

- **極低輸入延遲**：

  每次按鍵需在 ~50–100ms 內回饋。OT 讓 WebSocket 直達單一序列器（owner），排序＋transform 後立即回送，延遲最小。

- **決定性總序（deterministic total order）**：

  變更建議、註解錨點、樣式套用皆仰賴穩定順序；OT 先定序再廣播，邏輯清楚。



流程如下：

1. Client 透過 API Gateway 請求開始編輯文件。
2. API Gateway 回傳 Document Service 中對應的 socket server 位址，client 與其建立 WebSocket connection。
3. 連線建立後，使用者開始將 edits 傳送至 Document Service。
4. Document Service 對 edits 執行必要的 OT，再將結果寫入 DB（此處稱為 operation store）。
5. 由於寫入吞吐量極高，可使用 Cassandra（針對 high write throughput 最佳化），以 `doc_id` 作為 partition key、`ts` 作為 sorting key。

3. 使用者可以即時更新其他使用者對同一份文件的修改

可分成兩種情境：

情境一：使用者剛進入文件開始編輯

當使用者剛上線時，Document Service 可將該文件的所有 ops 一次 push 給使用者。

buildmoat.org

由於同一文件的所有使用者皆連到同一台 server，可視為 edits 由單一來源單向發送。

情境二：多位使用者同時在線協作

當多位使用者同時編輯時，每次 server 更新 operation DB，都必須將該 edit **broadcast** 給其他使用者，以滿足即時協作體驗。

這裡有一個關鍵細節：**client 端也必須執行 OT**。

假設：

- User A 與 B 同時看到文件狀態 S1
- A 送出基於 S1 的 edit `Ea`
- B 送出基於 S1 的 edit `Eb`
- Server 看到順序為 `Ea → Eb`，並對 `Eb` 做 OT
- 但從 B 的角度，即時順序會是 `Eb → Ea`

因此，B 的 client 需要對 `Ea` 再做一次 OT，才能維持一致的本地狀態。

4. 使用者必須即時看到其他協作者的游標位置

此需求有幾個重要特性：

1. 只有「當下」的位置重要，過去的位置不需要保存
2. 使用者離線後，不需要再回報位置
3. 位置更新的 latency 必須非常低

基於上述特性，這類資料非常適合只存放在 memory：

1. 使用者在線時，每 100ms 將游標位置回報給 WebSocket server。
2. WebSocket server 將收到的游標資料 broadcast 給其他使用者。
3. 使用者離線時，WebSocket server 直接將該使用者的游標資料從 memory 移除。

深入探討（Deep Dives）

1. 我們要如何擴展到數百萬個 WebSocket 連線？

很明顯地，單一 server 無法同時處理數百萬甚至上千萬個 connections。因此，我們必須將 Document Service 水平擴展到多台 servers，讓每台 server 負責一部分文件與使用者。這裡有幾個關鍵問題需要思考：

buildmost.org

1. 要如何將使用者平均分配到各個 server？
2. 在分配完成後，當使用者連線時，要如何確保他們能被正確且平均地導向到 server？
3. 當我們新增 server 或某些 server 下線時，要如何重新分配 user connections？

Consistent hashing（一致性雜湊） 可以解決問題 1 與 3（細節可參考講義）：

它能將 users 均勻分配到各個 server，並在 server 加入或移除時，將需要重分配的 user connections 降到最低。

Service Registry（服務註冊中心，例如 Zookeeper、Consul） 則用來解決問題 2。

它們負責偵測 distributed system 中的 endpoints：server 上線時 register，下線時 deregister；背後通常透過 quorum consensus 來確保系統的強一致性。

當使用者嘗試連線時，流程如下：

1. Registry 會將 online nodes 與 ring_config 推送（push）給 Gateway / Servers；雙方在本地使用 `hash(doc_id, healthy_nodes)` 計算該文件的 owner。
    1. 每個 document server 也需要維護 membership，以便在 ownership 變更時進行文件 handoff。
2. Client 發送 HTTP request 至 API Gateway，Gateway 依照 ownership 將 client 導向對應的 document server。
3. 使用者成功連線到正確的 server 後，連線會被升級為 WebSocket。
4. Document server 從 operation store 載入 snapshot + ops，並在使用者開始編輯前先傳送給 client。

2. 我們要如何控制儲存成本（storage）？

在擁有數十億份文件的情境下，必須謹慎設計 storage 策略。

僅透過選擇 OT 而非 CRDTs，就已經能將 storage 需求降低一個數量級；但即使如此，若每份文件為 100KB，總量仍高達 100TB。

更糟的是，若某份文件累積了數百萬個 operations，則每個新加入的 client 都必須接收並重放所有 ops，這在效能與體驗上都不可接受。

一個非常自然的解法是：**定期對 operations 進行 snapshot / compaction（壓縮）**。

核心想法是：除了版本管理需求外，我們其實不需要永久保存所有 ops，而是可以將多個 ops 合併成較少的操作，以節省儲存與處理成本。

buildnoat.org

Compaction 流程如下：

1. 在 `server_seq = S` 時，對「上一次 snapshot 與 S 之間的 ops」進行 compaction。
2. 範例：`insert("H")` + `insert("i")` → `insert("Hi")`
3. 當 compaction 在 `server_seq = S` 完成後，新加入的 clients 只需接收 `snapshot@S + ops(S+1…head)`，而不必重播完整歷史。
4. 舊的 ops（≤ S）可移至 cold storage，或在短期保留後進行 GC。
5. 常見的觸發條件：
    - 每 N 個 ops（例如 2k–10k）
    - 每 T 分鐘（例如 5–15 分鐘）
    - 自上次 snapshot 起的 delta 大小超過門檻（例如 1–4MB）
    - 或在 ownership handoff 時

Compaction 可在 doc server 線上執行，或由獨立的 offline process 處理；實務上可採用 hybrid 模式：

1. Online doc server 針對同一 client 的小批次 ops，每 50–100ms 進行輕量 compaction。
2. Offline worker 週期性進行 durable snapshot compaction，並負責 GC。

3. 我們要如何支援 Viewer Mode？

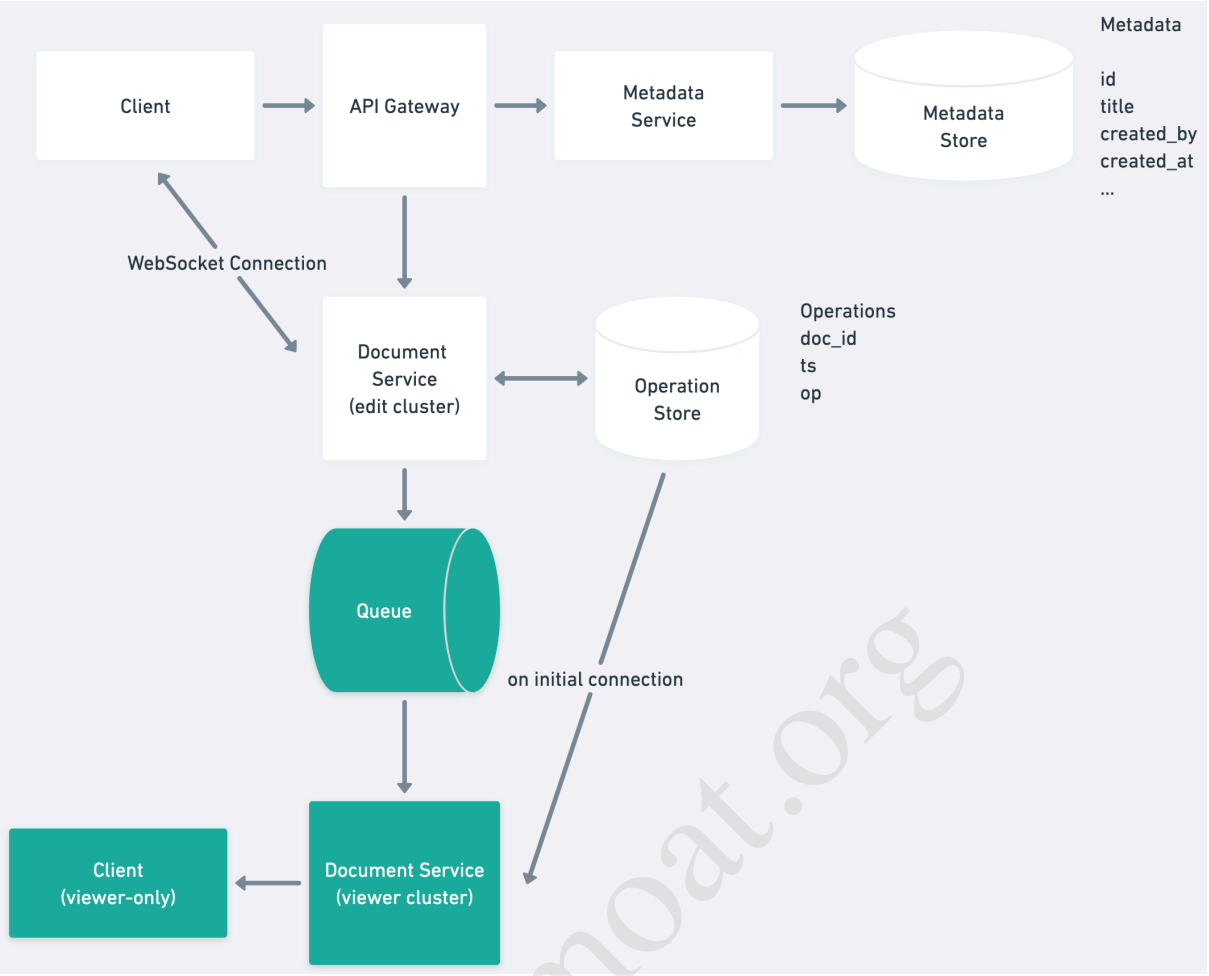
Google Docs 支援 viewer mode：使用者只有觀看權限，無法進行線上編輯。

要支援這種模式，需要考慮以下特性：

1. Viewer mode 使用者不會上傳任何 edits → 不一定需要與可編輯使用者在同一台 server。
2. 但 viewer 仍需要即時接收其他使用者的更新 → document service 仍需即時推送 edits。

一個可行解法是：**建立一個專門給 viewer mode 的 Document Service cluster**。

buildmoat.org



buildmoat.org

流程如下：

1. 當 collaborators 推送 edits 時，負責編輯的 document service（edit owner）會對 edits 執行 transformation、指派 `server_seq`，並將其持久化到 operation store。
2. 持久化完成後，edit owner 會將 ops 發佈到一個 durable event queue（例如 Kafka），由 viewer cluster 中的 viewer-owner 消費。
   - Queue 提供的好處：
     - **解耦（Decoupling）**：viewer cluster 的故障或 backpressure 不會影響 edit servers，且可獨立擴展。
     - **順序與耐久性（Ordering + durability）**：所有 viewers 看到的 ops 順序，與 edit server 持久化的順序完全一致；viewer-owner 可在重連時從 offset 進行 replay。
   - Viewer-mode clients 不需要執行 local OT，只需依序套用 ops。

- 由於 viewer cluster 僅做單向廣播（server → client），可改用 SSE（Server-Sent Events），其開銷比 WebSocket 更低，單一節點可服務更多 clients。（若需要游標或 presence，可走獨立通道）
- 從概念上看，viewer clients 會感覺像是「只有一位作者在對他們廣播 edits」。

3. Viewer client 重新連線時，會提供最後看到的 `resume_seq`；server 回傳 `snapshot@S + ops(S+1…head)`。

   若 gap 過大，server 可直接回傳 `snapshot@head` 加上一小段 tail ops。

buildmoat.org